LUMA GUARD: A Smart Safety System for Flame Detection and Distance Monitoring

MINI PROJECT REPORT

Submitted by

RAHUL P	**111924EC01125**
RUDRA KUMAR M	**111924EC01136**
RONALD REX C H	**111924EC01135**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

S.A. ENGINEERING COLLEGE, CHENNAI 600 077

(Autonomous-Institute Level Research Centre - Affiliated to Anna University)

ANNA UNIVERSITY : CHENNAI 600025

NOVEMBER 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this Project Report **"LUMA GUARD: A Smart Safety System for Flame Detection and Distance Monitoring"** is the bonafide work of **"RAHUL P (111924EC01125), RUDRA KUMAR M (111924EC01136), RONALD REX C H (111924EC01135)"** who carried out the project work under My supervision.

SIGNATURE

Dr. B.R. TAPAS BAPU, M.E., Ph.D.,
PROFESSOR
HEAD OF THE DEPARTMENT
Department of Electronics and Communication Engineering
S.A. Engineering College
Chennai-600077.

SIGNATURE

Ms. G. VASUMATHI, M.E.,
ASSISTANT PROFESSOR
SUPERVISOR
Department of Electronics and Communication Engineering
S.A. Engineering College
Chennai-600077.

Submitted for the End Semester Practical Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We owe a great many thanks to many people who helped and supported us during the completion of our project. We take this opportunity to express our profound gratitude and deep regards to our Founder (Late) **Thiru. D.SUDHARSSANAM, M.L.A,** our Chairman **Thiru. D.DURAISWAMY,** our Secretary **Thiru. D. DASARATHAN** and our correspondent **Shri. S. AMARNAATH, M. Com.,** our Director **Shri. D. SABARINATH** for their exemplary guidance, monitoring and constant encouragement throughout the course of the thesis.

We are extremely thankful to our Principal **Dr. S. RAMACHANDRAN, M.E., Ph.D.** We are indebted to **Dr. B. R. TAPAS BAPU, M.E., Ph.D.,** Professor & Head, Department of Electronics and Communication Engineering, for his valuable guidance and useful suggestions during the course of the project.

We would like to extend our heartfelt thanks and gratitude to our supervisor **Ms. G. VASUMATHI, M.E.**, Assistant Professor, and our project coordinators **Ms. S. SHARON SWEETI, M.E.,** Assistant Professor from the Department of Electronics and Communication Engineering, for their helpful guidance and valuable support given to us throughout the project. Further, we sincerely express our gratitude to our beloved parents and to all those who contributed to the successful completion of this project. Their constant support and encouragement throughout the project helped us complete this work with confidence and success.

SUSTAINABLE DEVELOPMENT GOALS



3 - GOOD HEALTH AND WELL-BEING

LumaGuard supports local hazard awareness on electronics workbenches and educational laboratories. It enhances operator safety through early visual and audible warning cues and functions as an educational prototype.

9 - INDUSTRY, INNOVATION AND INFRASTRUCTURE

The project demonstrates structured embedded firmware design on low-cost 8-bit hardware, implementing non-blocking state scheduling, exponential signal filtering, and persistent event logging.

11 - SUSTAINABLE CITIES AND COMMUNITIES

The system explores localized micro-monitoring frameworks for safer built environments, providing a reproducible blueprint for benchtop hazard tracking and work-zone safety sub-systems.

ABSTRACT

LumaGuard is an *Arduino Nano*-based flame-event and work-zone monitoring prototype developed around a low-cost infrared flame sensor, an *HC-SR04* ultrasonic module, an *SSD1306 OLED* display, indicator LEDs, a buzzer and a single pushbutton. The design objective is to improve on a simple sensor-to-buzzer circuit by making detection decisions more deliberate, alarm behaviour easier to interpret and recovery behaviour testable. The *DHT22* temperature-humidity module was excluded after repeated failed data-read tests; it is not part of the final system.

The flame module provides digital and analog outputs. The digital output supplies the module comparator state, while the analog output is used to observe relative changes in the sensor response. Firmware applies persistence and separate trigger/clear thresholds before treating a flame-like response as a confirmed event. The ultrasonic module independently monitors the distance to a reflecting object in a defined work zone. Its alert threshold is **30 cm**, with a separate clear threshold used to reduce rapid switching around the boundary. The ultrasonic measurement is not interpreted as flame distance or proof of human presence.

The system is organized as a small embedded state machine. It controls the green and red LEDs, generates short buzzer pulses with a cadence related to the selected hazard, displays sensor and alarm state, and supports short-press acknowledgement and long-press arming/disarming. A nonvolatile event-start counter can be stored in the Nano *EEPROM*. All processing is local, with *Serial Monitor* diagnostics available for calibration and testing.

The completed integrated version was reported as having passed end-to-end functional testing for its included functions. Specific response times, false-trigger rates and distance-error values were not supplied for this report and are therefore left as clearly marked fields for measured results. **LumaGuard** is intended as an educational prototype and is not a certified fire alarm or a substitute for an approved safety system.

TABLE OF CONTENT

CHAPTER	TITLE	PAGE
	BONAFIDE CERTIFICATE	i
	ACKNOWLEDGEMENT	ii
	SUSTAINABLE DEVELOPMENT GOALS	iii
	ABSTRACT	iv
	TABLE OF CONTENTS	v
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
	LIST OF ABBREVIATIONS	x
1	**INTRODUCTION**	1
1.1	PROJECT BACKGROUND AND OVERVIEW	1
1.2	PROBLEM STATEMENT AND NEED	2
1.3	OBJECTIVES, SCOPE AND CONTRIBUTION	3
2	**LITERATURE SURVEY**	4
2.1	LOW COST FLAME SENSING	4
2.2	ULTRASONIC WORK ZONE SENSING	5
2.3	FILTERING AND ALARM MANAGEMENT	6

2.4	COMPARISON AND ENGINEERING GAP	7
3	**SYSTEM DESIGN AND METHODOLOGY**	8
3.1	REQUIREMENTS AND ARCHITECTURE	8
3.2	SENSOR ROLES AND DATA FLOW	9
3.3	ALARM-STATE MODEL	11
3.4	DECISION RULES AND THRESHOLDS	12
3.5	PIN MAPPING	13
4	**HARDWARE AND SOFTWARE IMPLEMENTATION**	14
4.1	ARDUINO NANO AND FLAME MODULE	15
4.2	HC-SR04 DISTANCE MODULE	16
4.3	OLED, LEDS, BUZZER AND BUTTON	17
4.4	SOFTWARE ENVIRONMENT	18
4.5	FILTERING AND ALERT TIMING	19
4.6	OPERATOR CONTROLS AND EVENT COUNTER	20
4.7	ASSEMBLY, POWER AND COST	22
5	**TESTING, RESULTS AND DISCUSSION**	23
5.1	TEST OBJECTIVE AND METHOD	23
5.2	FLAME BASELINE AND CALIBRATION	24
5.3	HC-SR04 DISTANCE VALIDATION	25
5.4	INTEGRATED FUNCTION TESTS	26

5.5	DISCUSSION AND MEASUREMENTS	27
6	**CONCLUSION AND FUTURE SCOPE**	28
6.1	CONCLUSION	28
6.2	LIMITATIONS AND FUTURE SCOPE	28
6.3	FINAL REMARK	28
	REFERENCES	30
	APPENDIX	31

LIST OF FIGURES

FIGURE	TITLE	PAGE
3.1	OVERALL SYSTEM ARCHITECTURE	9
3.2	SIMPLIFIED ALARM STATE MODEL	11
3.3	EXISTING NANO PIN ASSIGNMENT	13
4.1	ARDUINO NANO BOARD	14
4.2	HC-SR04 MODULE	16
4.3	COMPLETED PROTOTYPE PHOTO	17
4.4	MAIN FIRMWARE FLOW	19
4.5	WIRING RELATIONSHIP DIAGRAM	21
5.1	OLED DISPLAY LAYOUT	27

LIST OF TABLES

TABLE	TITLE	PAGE
2.1	SENSOR AND PROCESSING COMPARED	7
3.1	FUNCTIONAL REQUIREMENTS	8
3.2	INPUT/OUTPUT MAPPING	13
4.1	HARDWARE SUMMARY	14
4.2	KEY FIRMWARE PARAMETERS	18
4.3	ADDITIONAL COMPONENT COST POSITION	22
5.1	CALIBRATION RECORD TEMPLATE	24
5.2	ULTRASONIC DISTANCE TEST RECORD	25
5.3	INTEGRATED VERIFICATION MATRIX	26
5.4	PERFORMANCE METRIC RECORD	27

LIST OF ABBREVIATION

ADC	-	Analog-to-Digital Converter
EEPROM	-	Electrically Erasable Programmable Read Only Memory
GPIO	-	General Purpose Input/Output
HC-SR04	-	Ultrasonic distance measurement module
IDE	-	Integrated Development Environment
LED	-	Light-Emitting Diode
MCU	-	Microcontroller Unit
PWM	-	Pulse-Width Modulation
SRAM	-	Static Random-Access Memory
UART	-	Universal Asynchronous Receiver-Transmitter

CHAPTER 1

INTRODUCTION

1.1 Project Background and Overview

Small embedded monitoring systems can provide local, immediate indications of changing conditions without depending on a network connection or a remote service. In an electronics workbench or teaching laboratory, a simple status device can help an operator notice a flame-like optical response or an object entering a defined work zone. The value of such a device depends not only on the sensor, but also on how the firmware interprets noisy inputs, communicates uncertainty and manages the resulting alert.

Luma Guard is an Arduino Nano-based educational prototype that combines an infrared flame-sensor module with an HC-SR04 ultrasonic distance module. Its user interface consists of a 128 × 64 SSD1306 **OLED**, green and red **LEDs**, a piezo buzzer and one pushbutton. The controller samples the sensor signals, applies configured thresholds and timing rules, updates a small set of hazard-state variables, and presents the current interpretation locally. A non-volatile event-start counter is stored in **EEPROM**. The DHT22 temperature-humidity sensor was tried during development, failed to provide dependable readings, and is excluded from the final working design.

The two sensing channels have distinct meanings. The flame module responds to optical energy within its spectral response; its digital output reflects the onboard comparator state and its analog output is a relative electrical reading. The HC-SR04 estimates the distance to a reflecting object from an acoustic echo. It does not detect flames or identify people. **Luma Guard** therefore combines complementary observations rather than treating the sensors as redundant measurements.

1.2 Problem Statement and Need

A direct connection from a raw sensor output to a buzzer can react to brief transients, produce repeated switching near a threshold, and provide little information about whether an event is being checked, confirmed, acknowledged or unavailable because a reading is stale. The project addresses this problem at prototype scale by using temporal persistence, separate trigger and clear boundaries, simple operator controls, and a display that distinguishes relevant operating states.

The objective is not to certify a fire protection instrument. It is to demonstrate how inexpensive modules can be integrated into a more interpretable embedded monitoring node, with explicit limitations and a testable state model. The system is intended for supervised educational demonstration and must not replace a certified alarm or established laboratory safety procedure.

1.3 Aim and Objectives

The aim is to design, integrate and document a low-cost local monitoring prototype using the existing Arduino Nano and peripheral modules. The objectives are to:

- Reuse the existing controller, flame module, HC-SR04, **OLED**, **LEDs**, piezo buzzer and single pushbutton.
- Confirm persistent flame-like candidates instead of accepting every brief raw transition.
- Implement a configured work-zone alert boundary of 30 cm and a separate clear boundary above 33 cm to reduce state chatter.
- Provide readable operating status and active alert information on the **OLED**.
- Provide short-press acknowledgement and a long-press arming/disarming control using one button.
- Generate audible pulses whose period depends on the configured hazard logic, and keep display and serial updates on timed schedules.
- Record hazard-start events in **EEPROM** and check the integrated behaviours through a defined functional test matrix.
- Exclude the failed DHT22 from the final circuit and operational firmware, while documenting its troubleshooting history.

1.4 Scope of the Project

The scope includes local optical flame-response monitoring, ultrasonic work-zone distance monitoring, signal filtering and persistence checks, alert-state handling, **OLED** telemetry, indicator outputs, button debouncing, serial diagnostics and a small non-volatile event counter. The scope does not include cloud connectivity, remote notification, person identification, calibrated flame intensity, certified fire detection, industrial safety compliance, or quantified metrological performance beyond evidence recorded in the project material.

1.5 Engineering Contribution

The principal contribution is the integration of familiar modules with explicit state and recovery rules on an 8-bit microcontroller. Instead of treating a sensor transition as the entire decision, the firmware retains candidate, confirmed, acknowledged and data-freshness information. The interface communicates a small vocabulary of states, and a newly activated hazard can be unacknowledged independently of an already silenced source. This is a useful demonstration of embedded decision logic; it is not evidence that a low-cost prototype achieves safety-instrument performance.

1.6 Constraints, Assumptions and Safety Boundary

The design reuses the available hardware and keeps mandatory additional component cost at zero in the documented build. The reported classic Nano configuration uses the **ATmega328P** family, 5 V logic and the established digital, analog and I²C connections documented in Chapter 3. Thresholds are configuration values, not universal sensor specifications; they must be reviewed for the exact module, installation angle and ambient environment.

Optical response can be affected by sunlight and other infrared sources, while ultrasonic readings depend on the target surface, angle and echo path. The **prototype** is suitable only for controlled, supervised testing. The report proceeds from technical context to system design, implementation, functional verification, conclusions and the complete source listing.

The HC-SR04 routine calls *pulseIn()* with a 25,000 µs timeout and can block the loop. Thus, the firmware is not fully non-blocking; precise latency is not claimed.

CHAPTER 2

TECHNICAL REVIEW AND DESIGN CONTEXT

2.1 Low-Cost Optical Flame Sensing

Low-cost flame modules commonly combine a photodetector and an onboard comparator. The comparator output (**DO**) changes state when the detected signal crosses a module-adjustable threshold, while an analog output (**AO**), where provided, exposes a relative voltage for analog-to-digital conversion. The exact wavelength response, circuit and logic polarity vary by module; therefore the physical model and seller datasheet should be checked rather than assuming all modules behave identically. The supplied project record describes **DO** as active **LOW** and uses D9 for **DO** and A0 for **AO**. A published guide illustrates the general low-cost analog flame-module class, but it does not establish the exact variant installed in this project [6].

An analog reading is not a calibrated measure of flame intensity or distance. It depends on detector response, module design, flame type and position, field of view, ambient infrared energy, potentiometer adjustment and supply conditions. The correct report language is therefore "relative flame-sensor response" or "flame-like optical response," unless a separate calibration establishes a stronger interpretation. The project uses initial **AO** trigger and clear thresholds of 800 and 850 respectively, while the digital comparator can independently flag a candidate. These values belong to the current firmware configuration and require environment-specific verification.

2.2 Ultrasonic Work-Zone Monitoring

The **HC-SR04** emits a short ultrasonic burst and measures the duration of the returning echo pulse. The common calculation estimates distance from echo duration and the speed of sound, dividing the round trip travel distance by two. The source module documentation describes a short trigger pulse and a pulse width echo output; the project code uses a 10 μs trigger pulse and rejects zero duration results and values outside its configured 2–400 cm validity range.

The sensor reports the location of a reflecting surface along its acoustic path. It cannot determine whether that surface is a person, an approaching hazard, or any particular object type. Soft materials, angled surfaces, narrow objects and surrounding acoustic reflections may reduce echo reliability. The project therefore treats distance as **work zone** proximity telemetry only, not as evidence of a flame or human presence.

2.3 Filtering, Persistence and Hysteresis

An exponential moving average (**EMA**) reduces short-term variation by combining the most recent valid measurement with the prior filtered value. In the supplied firmware, the new distance estimate is weighted at 0.40 and the previous estimate at 0.60. This smooths jitter but also introduces some lag: a filter that is stronger at suppressing variation responds more slowly to real transitions. Filter settings should therefore be understood as a trade-off, not a free improvement.

Persistence requires a candidate condition to remain present for a configured interval before the system confirms it. The flame candidate is required to remain active for 250 ms. Hysteresis uses different entry and exit thresholds so that small variations around one boundary do not repeatedly toggle the state. For proximity, the current firmware enters zone-danger at or below 30 cm and clears only after the filtered distance exceeds 33 cm for the 350 ms clear hold. Flame recovery requires the clear inputs (**DO HIGH** and **AO** at or above 850) to remain valid for 1,200 ms. These timings are code parameters, not independent measurements of system response.

2.4 Local Display and Audible Annunciation

A small **OLED** can communicate more context than a single indicator by showing arming mode, flame status, distance and alert status. The Adafruit SSD1306 driver supports monochrome **OLED** modules based on the SSD1306 controller and depends on the Adafruit GFX library for drawing primitives. The **LEDs** and buzzer provide at a glance and audible indication. In the firmware, the green **LED** is used for an armed, apparently clear state when the distance reading is fresh. The red **LED** and buzzer are activated for an unacknowledged hazard while armed. A short press silences currently active sources but leaves the hazard state visible; a newly active source can re-trigger an alert. Disarmed mode turns off physical annunciators while sensor evaluation continues.

2.5 Non-Volatile Event Logging

EEPROM preserves data across power removal on the classic AVR controller; Arduino's **EEPROM** library exposes operations for reading and writing stored values [5], [9]. The project stores a small hazard-start counter alongside a signature value. The signature allows the firmware to distinguish a previously initialized record from uninitialized or incompatible stored content. This counter provides a simple event count, not a timestamped history: it does not record event duration, sensor traces, ambient conditions or an auditable sequence of all measurements.

2.6 Comparison of Approaches

Single comparator driving a buzzer	Very simple circuit and easy initial demonstration.	Momentary transitions can create nuisance alerts; state context is limited.
Analog flame output only	Exposes relative change in the optical sensor response.	Requires module-specific threshold choice; it is not calibrated flame intensity.
Digital flame output only	Convenient binary comparator state.	Depends on module potentiometer and logic polarity; loses analog variation.
Ultrasonic distance only	Provides distance to a reflecting object.	Does not identify the object or detect flame.
Luma Guard combination	Separates optical and proximity flags, provides persistence, hysteresis and operator acknowledgement.	Still uses low-cost sensors; results depend on installation, calibration and controlled validation.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Design Requirements

Requirements connect each intended system function to a method of verification. They define observable behaviour rather than presumed reliability. The “reported outcome” column is reserved for the integrated functional status supplied for this project; it should not be interpreted as an independently audited accuracy or safety certification.

FR-01	Sample flame **AO** and **DO** and present the interpreted state.	Serial diagnostics and controlled sensor observation.
FR-02	Confirm a persistent flame-like candidate after the configured persistence interval.	Brief-transient versus sustained-candidate check.
FR-03	Activate the work-zone alert at the configured 30 cm entry boundary.	Ruler-positioned target test.
FR-04	Use a separate >33 cm clear boundary with a hold interval.	Move target across the boundary and observe recovery.
FR-05	Adjust audible pulse period according to active hazard logic.	Compare alert behaviour at configured test conditions.
FR-06	Acknowledge active hazards using a short pushbutton press.	Operate the button while an alert is active.
FR-07	Toggle **ARMED/DISARMED** with a 2-second long press.	Observe mode and physical outputs before and after press.
FR-08	Present mode, flame status, distance and alert text on the **OLED**.	Inspect display during clear, alert and no-echo conditions.
FR-09	Retain the event-start counter across power cycles.	Trigger an event, power-cycle, then inspect serial count.

3.2 Overall Architecture

The hardware is organized around the Arduino Nano, which collects the two sensor inputs and drives the display, indicator **LEDs** and buzzer. The pushbutton is read through a pull-up input. The controller maintains the sensor estimates and hazard flags, applies the timing thresholds, updates the acknowledgement mask, and then calculates the required outputs.

OLED and serial output are refreshed on separate time intervals. The event counter is updated only when a hazard bit becomes newly active, rather than continuously while the same hazard remains active.

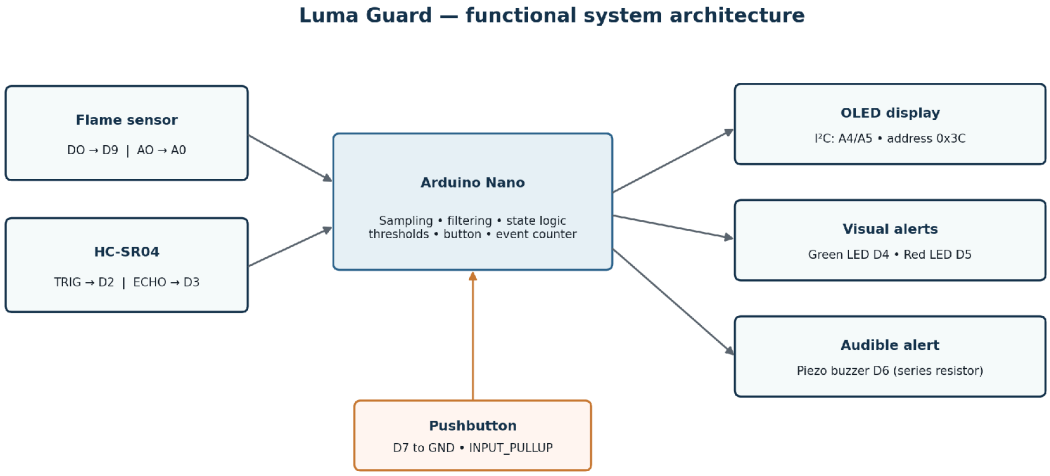


Figure 3.1. Functional architecture and documented Nano signal connections.

3.2.1 Processing and Data Flow

The processing sequence is intentionally kept understandable: the button is checked; when the sensor interval expires, the firmware performs a distance read, updates the distance and flame state, and calculates the active hazard mask; display and serial telemetry are updated when their own intervals expire. **EEPROM** is written when newly active hazard bits are detected.

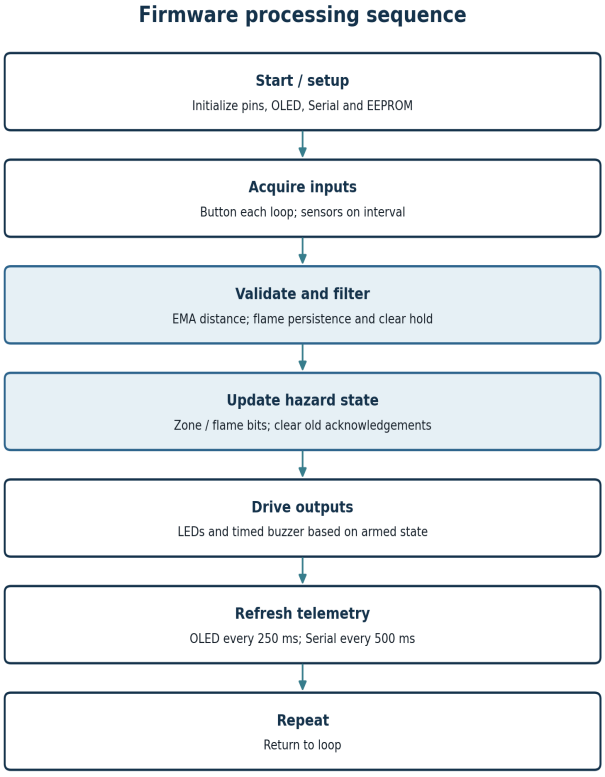


Figure 3.2. Firmware processing sequence. Sensor acquisition includes a bounded pulseIn() wait.

3.3 Sensor Roles and Data Interpretation

Signal or Input	What it Represents	What it Does Not Establish
Flame **DO** (D9)	Digital comparator state from the optical module; documented as active **LOW**.	Does not prove fire in every environment and can be affected by ambient **IR**.
Flame **AO** (A0)	Relative analog response from the same flame module.	Not calibrated flame intensity, flame distance or an independent detector.
HC-SR04 echo (D2/D3)	Time of flight to a reflecting target in the acoustic field.	Does not identify a person or determine the cause of proximity.
Pushbutton (D7)	User action duration used for acknowledgement or mode toggle.	Cannot confirm that the physical hazard has disappeared.
OLED/LED/buzzer	Local presentation and annunciation of computed states.	Cannot improve the underlying accuracy of a sensor by itself.

Flame and distance inputs must not be interpreted as a single physical measurement. They are independent hazard flags: the flame flag represents a persistent optical candidate, and the zone flag represents the configured proximity state. When both are active, the buzzer logic selects the shorter of their computed periods. A display state such as **SAFE** is a statement about currently configured flags and input freshness, not proof that the physical environment is hazard-free.

3.4 Alarm State and Acknowledgement Model

The implementation uses a small set of Boolean states and hazard bitmasks rather than a single monolithic enumerated state variable. The user-visible status is derived from armed mode, unsilenced hazard flags, the active hazard mask, the current flame candidate and the freshness of the latest valid distance reading. This distinction matters: acknowledgement affects sound and indicator outputs, but it does not clear the underlying hazard flag or stop monitoring.

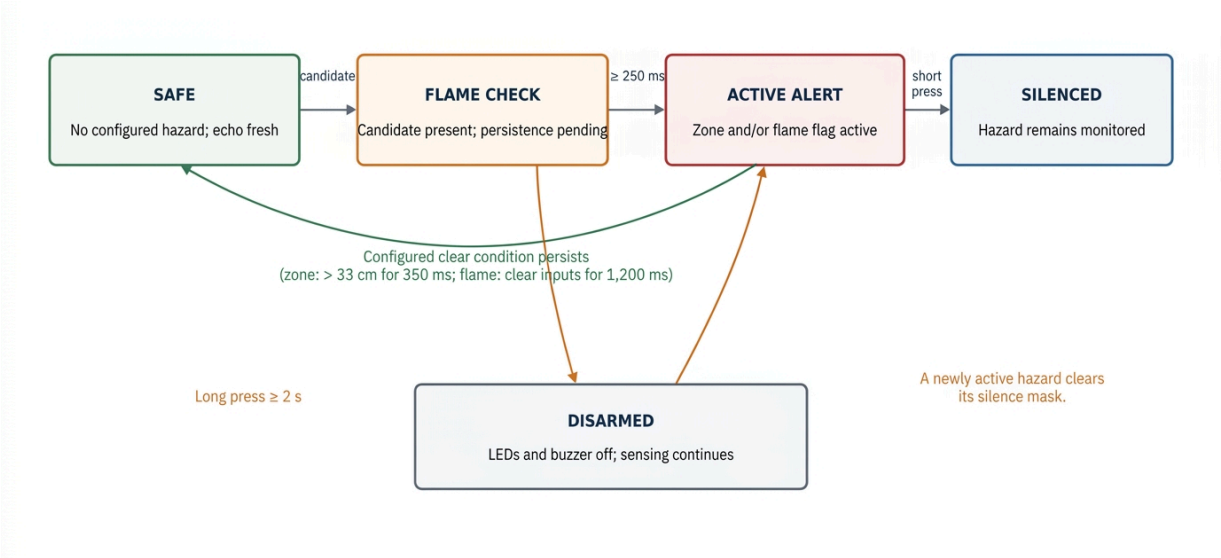


Figure 3.3. Conceptual operating-state and acknowledgement transitions derived from the supplied firmware.

3.4.1 Candidate Confirmation and Recovery

A flame candidate occurs when **DO** is **LOW** or **AO** is at or below 800. The candidate timer starts on the first detected candidate; a confirmed flame state is set after the candidate remains active for 250 ms. When the candidate disappears, the candidate timer resets. A previously confirmed flame state clears only when **DO** is **HIGH** and **AO** is at least 850 continuously for 1,200 ms. Values between the analog boundaries do not independently satisfy the clear condition, which helps prevent rapid switching.

The zone state is set when a valid filtered distance is at or below 30.0 cm. Once active, it is cleared only after the filtered distance exceeds 33.0 cm for at least 350 ms. When no echo is returned, the display can mark the distance as stale after 800 ms without a valid reading. The firmware separately clears an active zone hazard after 1,200 ms without a valid distance, so "unknown" display freshness and the clearing of the prior hazard flag are not the same event.

3.5 Configured Decision Parameters

Parameter	Value in Supplied Sketch	Meaning
Zone alert entry	≤ 30.0 cm	Filtered distance activates the zone-danger flag.
Zone clear boundary	> 33.0 cm	Clear condition must exceed the higher boundary.
Distance **EMA**	0.60 previous + 0.40 new	Smooths valid distance readings.
Flame candidate	AO ≤ 800 OR DO **LOW**	Starts persistence confirmation.
Flame clear condition	AO ≥ 850 AND DO **HIGH**	Both inputs must indicate clear.
Flame confirmation	250 ms	Minimum candidate persistence before confirmation.
Flame clear hold	1,200 ms	Stable clear interval before clearing confirmed state.
Zone clear hold	350 ms	Stable beyond-boundary interval before clearing zone alert.
Distance display freshness	800 ms	Older valid echo results are displayed as stale/NO ECHO.
No-echo zone clear	1,200 ms	Clears a prior active distance hazard after prolonged invalid echo.
Button debounce	35 ms	Rejects short transitions during button operation.
Long press	2,000 ms	Toggles ARMED/DISARMED mode.
Buzzer pulse ON	80 ms at 2,400 Hz	Defines audible pulse tone duration and frequency.
Beep period range	180–900 ms for zone logic; flame path uses 650–180 ms	Configured pulse-start interval bounds used by urgency mapping.

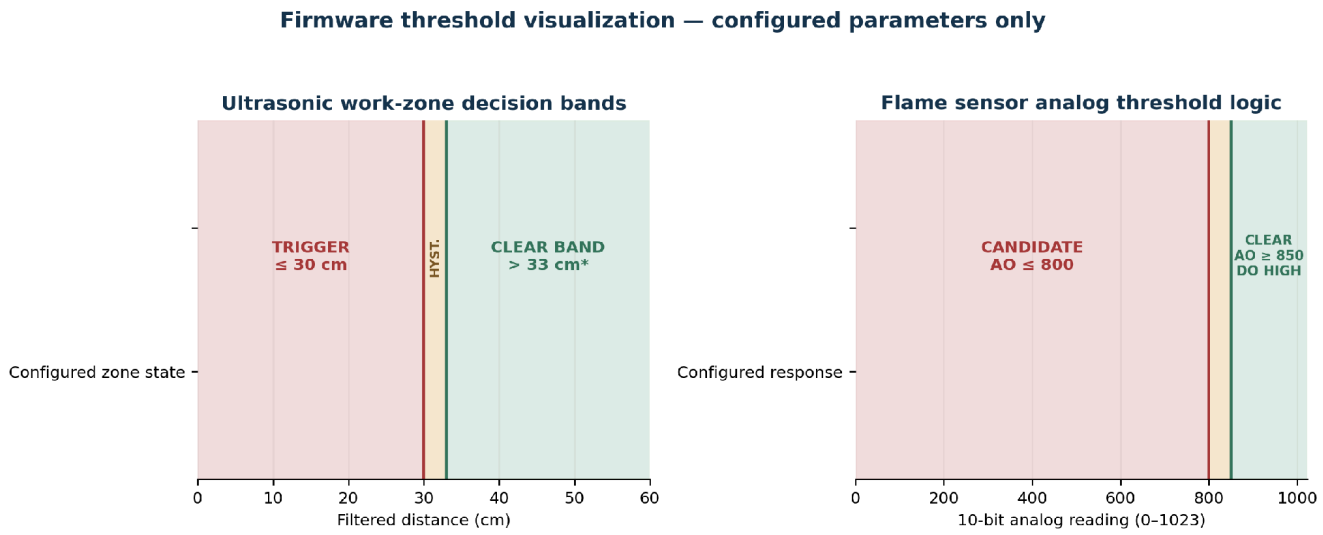


Figure 3.4. Visual summary of configured threshold bands. This is a logic diagram, not a plot of measured performance.

3.5.1 Distance Calculation

For an HC-SR04 echo pulse, the code converts duration to centimetres using distance = duration × 0.0343 / 2, where duration is in microseconds and 0.0343 represents the approximate sound travel distance in centimetres per microsecond used by the sketch. Dividing by two accounts for the outward and return paths. The conversion is an estimate under ordinary sound-speed assumptions; temperature, target geometry and acoustic conditions can affect the real-world result. The sketch rejects missing echoes and distances outside 2–400 cm.

3.5.2 Alert Cadence Mapping

The zone cadence maps the filtered distance from the 30 cm boundary toward the configured 2 cm lower calculation limit, producing a period that shortens from 900 ms toward 180 ms as the object approaches. The flame cadence maps the relative analog reading between references of 1,000 and 400 to an alert period from 650 ms toward 180 ms. If both unacknowledged hazards are active, the smaller computed period is selected. These are deterministic mappings from firmware constants and the current sensor readings; they do not constitute measured proof of perceptual urgency or a calibrated severity scale.

3.6 Pin Mapping and Wiring Relationships

Device	Signal / Connection	Nano Pin and Notes
HC-SR04	TRIG / ECHO	D2 / D3
Green **LED**	Anode through existing current-limiting resistor	D4
Red **LED**	Anode through existing current-limiting resistor	D5
Piezo buzzer	Signal through the documented existing 1 kΩ series resistor	D6
Pushbutton	D7 to **GND** when pressed; *INPUT_PULLUP*	D7
Flame module	**DO / AO**	D9 / A0
SSD1306 **OLED**	SDA / SCL; configured I²C address 0x3C	A4 / A5
DHT22	Disconnected after failed data-read tests	Excluded from final design

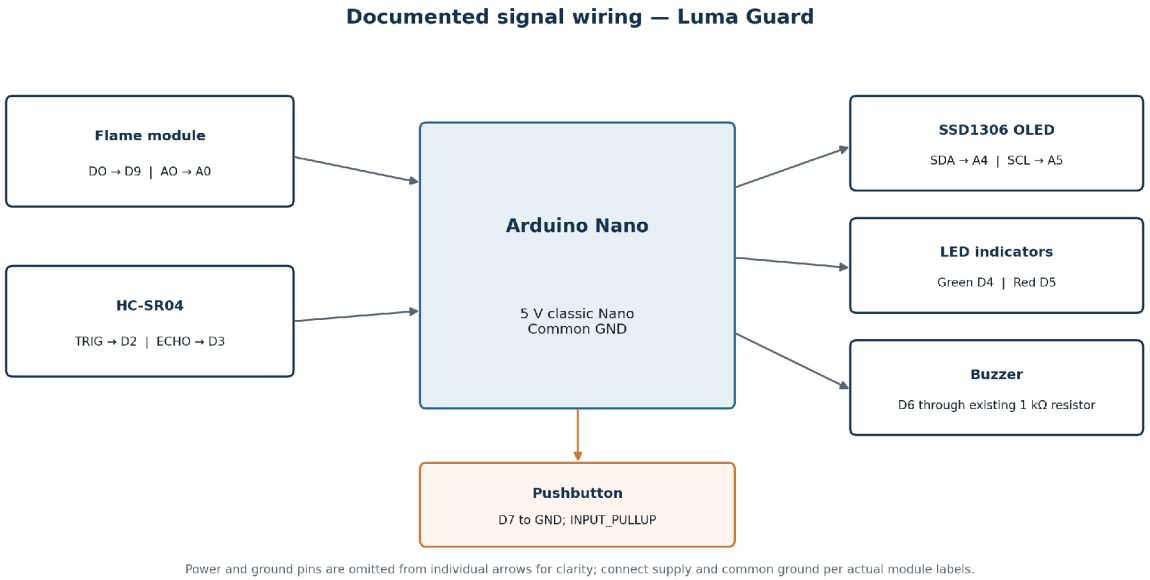


Figure 3.5. Wiring relationship diagram based on the pin assignments documented in the source report.

Verify each module's supply, ground and pin order before power-up. Retain the existing resistors; the DHT22 is disconnected and excluded from the final circuit.

3.7 Data Quality, Fault Indication and State Priority

The firmware uses input freshness and internal state flags to avoid presenting every condition as a simple clear/alert binary. These distinctions are important during a demonstration because an unknown distance, an unconfirmed optical candidate and a silenced confirmed hazard have different meanings. They also determine which display message is selected and whether the physical outputs should operate.

Condition	Firmware Interpretation	Output / Display Implication
No recent valid echo	Distance is stale when the most recent valid reading is older than the configured freshness interval.	**OLED** may show **NO ECHO** and alert text **ZONE UNKNOWN** when no other higher-priority state applies; green **LED** is not asserted.
Flame candidate only	One of the raw flame criteria is active, but persistence has not confirmed it.	Flame text is **CHECK**; the system does not yet label it **DETECTED**.
Confirmed but acknowledged hazard	Hazard flag remains active while its source bit is in the silence mask.	**OLED** shows **SILENCED** unless a newly active source or disarmed mode changes the status; audio is stopped.
New hazard source after acknowledgement	A newly active hazard bit is not present in the prior silence mask.	Its silence bit is cleared by mask maintenance and it can alert independently.
OLED initialization failure	The display driver did not initialize successfully.	The firmware reports an error on Serial; the display update routine exits when *oledReady* is false.
Disarmed mode	The armed flag is false; sensor evaluation continues.	**LEDs** and buzzer are off, and the **OLED** status is **DISARMED**.

The status hierarchy is an interface decision, not a ranking of real-world hazard severity. The firmware prioritizes **DISARMED**, then unacknowledged flame/zone combinations, **SILENCED** active hazards, flame **CHECK** and distance freshness. Because a missing echo can coexist with a previous zone alert for the no-echo clear interval, the display and active flag may temporarily communicate different aspects of the same measurement failure; this follows the code's separate freshness and clearing timers.

3.8 Engineering Trade-offs

Three trade-offs shape the implementation. First, filtering reduces visible jitter but delays state transitions and can smooth abrupt movement. Second, wider entry/clear separation reduces threshold chatter but means recovery is not immediate when a target crosses back over the entry point. Third, local audio and visual cues are inexpensive and do not need wireless infrastructure, but they do not provide a remote log or independent emergency notification. The settings chosen here are plausible configuration values for a controlled prototype; they have not been optimized across a statistically defined set of operating environments.

3.9 Requirement-to-Implementation Traceability

The traceability table connects each functional requirement to the implemented firmware area and the integrated test scenario used to check its behaviour. Function names refer to the supplied sketch; test IDs correspond to the matrix in Chapter 5.

Requirement	Implementation Element	Verification Case
FR-01	*updateFlameState()*; *readDistanceCm()*; Serial diagnostics	IT-01 to IT-03
FR-02	*FLAME_CONFIRM_MS* and *flameCandidateStartedMs*	IT-03
FR-03	*DISTANCE_ALERT_CM* and *distanceDanger* transition	IT-02
FR-04	*DISTANCE_CLEAR_CM* and *distanceClearStartedMs*	IT-02, IT-07
FR-05	*calculateDistancePeriod()*, *calculateFlamePeriod()*, *calculateBeepPeriod()*	IT-02 to IT-04
FR-06	*handleShortPress()* and *silencedHazardMask*	IT-05
FR-07	*handleButton()* and *LONG_PRESS_MS*	IT-06
FR-08	*getAlertStatusText()* and *updateOLED()*	IT-01 to IT-08
FR-09	*updateHazardMask()*, *loadEventCounter()*, *saveEventCounter()*	IT-09

CHAPTER 4

HARDWARE AND SOFTWARE IMPLEMENTATION

4.1 Arduino Nano Controller

The classic Arduino Nano configuration is based on the ATmega328P microcontroller and provides digital I/O, analog inputs, serial communication, I²C and **EEPROM** facilities. The project uses the Nano as the sole controller; no external computer is required during ordinary operation. The computer and Arduino IDE are used for editing, compilation, uploading and observing Serial Monitor diagnostics. The classic Nano's limited **RAM** and flash encourage concise state variables, fixed-size data and small display routines.

The firmware stores a limited number of floating-point distance values, timing timestamps, Boolean flags, a hazard bitmask and a small event-count record. It does not retain a long time series. This keeps the data model understandable for the project's scope, although the code's actual memory use and compiler output should be inspected in the selected board package if resource utilization is being reported quantitatively.

4.2 Flame Sensor Interface

The flame module's digital output connects to D9 and its analog output to A0. The code samples both values during the sensor-update routine. The candidate condition is true if the digital comparator reads **LOW** or the analog reading is at or below 800. The clear condition requires the comparator **HIGH** and analog value at or above 850. These thresholds are kept as named constants near the beginning of the sketch so that changes are centralized and visible.

The **AO** and **DO** paths arise from the same optical module and should not be described as two independent detectors. A comparator adjustment can alter **DO** behaviour, and optical response varies with view angle, distance and ambient infrared radiation. Calibration is therefore the procedure of observing the module's relative response in the intended supervised environment and verifying that the configured states behave consistently, not the establishment of a universal fire intensity threshold.

4.2.1 Practical Calibration Procedure

- Keep the DHT22 disconnected and confirm that the flame module shares a stable common ground with the Nano.
- Open Serial Monitor at the configured 9,600 baud and observe the digital and analog channels with no intentional flame source present.
- Check that the clear environment does not hold a flame candidate; note the ambient conditions rather than copying a previous **AO** number as a universal baseline.
- Under supervision, use a small controlled flame and a non-flammable arrangement to observe whether the candidate and confirmed states behave as expected.
- Confirm the persistence interval, visual indication, buzzer response and clear-hold behaviour. Stop if wiring, power or sensor behaviour appears unstable.

4.3 HC-SR04 Ultrasonic Interface

The TRIG input is connected to D2 and ECHO to D3. The sketch drives TRIG low briefly, raises it for 10 µs, lowers it, and measures the ECHO high pulse with *pulseIn()* and a 25,000 µs timeout. Zero duration is treated as a missing echo. Valid results are converted to centimetres and rejected when outside the configured range. The distance estimate is filtered using an **EMA** before entering the work-zone state logic.

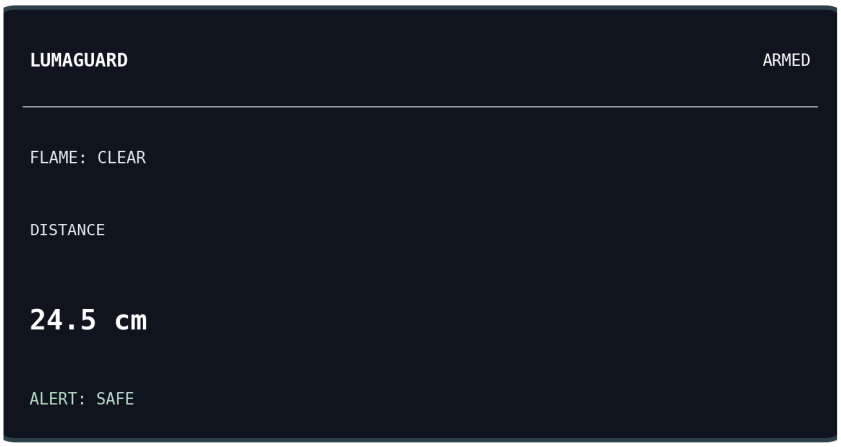
The code uses a 100 ms sensor-update interval, but *pulseIn()* can occupy the processor while waiting for the echo, up to the timeout. The implementation is consequently time-scheduled in many areas but not fully non-blocking. This distinction is relevant to any future timing optimization: an interrupt-driven or asynchronous echo measurement could reduce the longest waiting path, but it would require separate implementation and verification and is not claimed as part of this version.

4.3.1 Missing Echo and Stale Data

A missing echo must not be interpreted as a measured long distance. The code tracks the last valid distance time and reports **NO ECHO/ZONE UNKNOWN** when that value is no longer fresh under the 800 ms display-freshness rule. If a zone-danger flag is already active, the implementation applies a longer 1,200 ms no-echo interval before clearing that prior flag. This behaviour distinguishes uncertainty about the current measurement from a validated clear transition.

4.4 OLED, LEDs, Buzzer and Pushbutton

The SSD1306 **OLED** is configured as a 128 × 64 monochrome display at I²C address 0x3C. The screen shows the LUMAGUARD label, mode, flame status, distance or **NO ECHO**, and alert status. The illustrative layout below follows the text placement in the supplied firmware; the sample distance is a mock-up value intended to show formatting, not a recorded test result.



Illustrative 128 × 64 OLED layout; example values are mock-up text, not a test reading.

Figure 4.1. Illustrative OLED layout derived from the supplied display routine; the sample distance is not a measurement.

Green **LED** D4 indicates armed, clear operation only while distance data is fresh. Red **LED** D5 indicates an unacknowledged alert. A short press silences physical outputs but retains hazard status on the **OLED**; disarmed mode turns off **LEDs** and buzzer while sensor evaluation continues.

The buzzer signal is on D6 through the existing 1 kΩ series resistor identified in the project record. The sketch uses *tone()* at 2,400 Hz and limits an individual **ON** pulse to 80 ms. Pulse periods are calculated from the current zone distance and/or flame analog response, with the most urgent unacknowledged hazard determining the shorter period. The single button is connected between D7 and ground and uses *INPUT_PULLUP*. The code debounces the input for 35 ms, acknowledges active hazard sources on a short press, and toggles armed mode after a 2,000 ms hold.

4.5 Software Environment and Libraries

Software Element	Purpose in the Project
Arduino IDE / AVR board support	Edit, compile and upload the sketch; select the correct classic Nano processor option.
Wire	I²C communication with the OLED.
Adafruit_SSD1306	Initialize and render the SSD1306 display buffer.
Adafruit_GFX	Provide text and drawing primitives used by the display driver.
EEPROM	Read and write the small non-volatile event counter.
Serial Monitor	Display sensor values, modes, state transitions and event count at 9,600 baud.

The single-wire DHT library is intentionally absent because the DHT22 is not part of the final system. The **OLED** library's upstream documentation identifies Adafruit GFX as a required companion for graphics primitives [3], [4]. The exact library versions should be recorded from the actual installation if the report is later required to include software-version traceability; no version numbers are invented here.

4.6 State Handling, Timing and Outputs

The sketch uses *millis()*-based comparisons to schedule repeated sensor, display and serial activity. A 100 ms interval gates sensor updates, a 250 ms interval refreshes the **OLED**, and a 500 ms interval prints serial status. The hazard mask contains one bit for zone proximity and one for flame confirmation. When a bit becomes newly active, the **EEPROM** counter increments and the updated record is stored. The acknowledgement mask is then restricted to hazards that remain active; newly active sources therefore cannot be hidden indefinitely by a prior acknowledgement.

The distance filter is updated only after a valid measurement. The first valid reading initializes the filter; subsequent readings use the 0.60/0.40 previous/new weighting. The flame state separately tracks the candidate timer and clear timer. Output control examines armed mode and the unacknowledged mask before deciding whether to run the buzzer or light the **LEDs**. The **OLED** and serial messages are downstream views of these states rather than independent measurements.

4.7 EEPROM Event Counter

The event record contains a signature and a 32-bit hazard-start count. On setup, the program reads the record; if the signature does not match the expected value, it initializes the signature and counter and writes the initial record. A newly active bit in the calculated hazard mask increments the count and writes it to **EEPROM**. The record survives power cycles, subject to the microcontroller's **EEPROM** characteristics and the integrity of the write process. Frequent or continuous logging is not implemented, and the counter should not be mistaken for a timestamped incident log.

4.8 Assembly, Power and Cost

The project reuses the Arduino Nano, flame module, HC-SR04, **OLED**, indicator **LEDs**, piezo buzzer and one pushbutton. The final wiring uses the documented signal pin map in Chapter 3. The report records no mandatory additional component purchase in the tested build, so the mandatory additional-component cost is ₹0 based on the project inventory. Optional items such as a decoupling capacitor, perfboard or connectors are not included in the tested configuration and should not be represented as installed unless they were actually added.

Before power-up, inspect for a common ground, correct module orientation, secure jumper wires, retained **LED** current-limiting resistors, and a buzzer series resistor consistent with the assembled unit. Confirm the **OLED** address and the flame module output labels. An incorrect wiring assumption can damage modules or produce misleading state readings; therefore, the diagram is a documented signal map and not a substitute for inspecting the physical board and connector orientation.

4.9 Safety and Implementation Limitations

Luma Guard is an educational prototype, not a certified fire alarm or substitute for approved life-safety equipment. Its single optical module and ultrasonic sensor have field-of-view and environmental limits; it lacks an independent fault-tolerant detector, calibrated temperature channel and remote notification. The HC-SR04 read can block the loop for up to its configured timeout.

4.10 Commissioning and Troubleshooting

Commissioning should proceed from power and wiring checks to independent sensor observation, then to integrated output checks. It should not begin by assuming that a display or buzzer problem is a software-only fault. The following table summarizes safe, source-consistent diagnostic steps for common symptoms; it does not assert that a particular fault was observed in the final test.

Symptom	Checks in a Safe Diagnostic Order	Interpretation
OLED remains blank	Verify supply and ground; confirm SDA=A4 and SCL=A5 for the documented Nano; check configured address 0x3C; inspect Serial for **OLED** initialization failure.	If *oledReady* is false, display updates are skipped; sensing and serial diagnostics may still run.
Buzzer appears continuous	Confirm whether the firmware is repeatedly generating tone pulses; check the active and silenced hazard status in Serial/**OLED**; verify buzzer wiring and the existing series resistor.	A persistent alert may be caused by a sustained sensor state, not necessarily a stuck buzzer output.
Red/green **LEDs** do not match expectation	Check **ARMED/DISARMED** mode, the active hazard mask, acknowledgement state and whether distance data is fresh; then verify **LED** polarity and existing resistors.	Green is only asserted in the armed, clear and fresh-distance path; it is intentionally off during several other states.
Unexpected flame candidate	Observe **AO/DO** in Serial with no deliberate stimulus; inspect module orientation, ambient **IR** and comparator potentiometer; compare observations with configured thresholds.	Thresholds are module- and environment-dependent; do not treat a single **AO** number as universal.
ZONE UNKNOWN / NO ECHO	Check the target is within the module's useful geometry, the module orientation, TRIG/ECHO pins, common ground and the target surface.	A missing echo is invalid data, not proof that the zone is clear.
Counter does not appear to increment	Check that the transition created a newly active hazard bit; inspect Serial event-count output and repeat after a controlled power cycle.	The counter increments on a newly active hazard source, not on every sensor sample.

4.11 Demonstration Sequence

A clear demonstration should start with inspection and logging, not with a live hazard. Confirm the DHT22 is disconnected, verify the listed pin map and common ground, select the correct board and processor in Arduino IDE, upload the supplied sketch, open Serial Monitor at 9,600 baud, and verify that the **OLED** initializes. Observe a clear baseline before a controlled sensor stimulus. Exercise the zone boundary, flame persistence, acknowledgement, disarming and re-arming one at a time, then test combined inputs and the no-echo case. Finally, power-cycle the unit and inspect the event count. Record actual values only when they are observed during that session. Use supervised and approved lab precautions for any flame stimulus.

CHAPTER 5
TESTING, RESULTS AND DISCUSSION

5.1 Test Objective and Method

Testing was intended to establish that the included integrated functions respond to the defined scenarios: flame candidate persistence and recovery, ultrasonic proximity and clear hysteresis, escalating audible behaviour, button acknowledgement, arm/disarm operation, **OLED** telemetry, no-echo handling and **EEPROM** event counting. Tests should be performed on a stable bench with the documented pin map, the DHT22 disconnected, and the serial output available for state observation. Flame trials must be supervised and use a small controlled source in a non-flammable setup.

The project team reports that the final integrated version passed its end-to-end functional test scenarios. The source report also contains isolated **AO** and ruler-distance values, but no timestamped raw log or repeated-trial record sufficient to substantiate all of its detailed accuracy, latency, false-trigger and repeatability claims. Consequently, those unsupported metrics are not carried forward as established quantitative results. The configured thresholds are reported as firmware settings; they are not represented as metrological performance.

5.2 Flame Sensor Functional Verification

The flame verification sequence begins with the sensor in its ordinary ambient environment, then applies a controlled optical stimulus, observes the candidate state, checks the persistence rule, and removes the stimulus to observe recovery. The operational condition is based on the combination of **DO** and **AO** described in Chapters 3 and 4. Because module response varies with orientation, ambient infrared conditions and comparator adjustment, the **AO** thresholds should be treated as specific to the existing build.

Scenario	Expected Behaviour from Firmware	Reported Functional Outcome
Ambient / no candidate	**DO**/**AO** do not sustain a candidate; display reports **CLEAR** when no other condition dominates.	**PASS** as part of reported integrated tests.
Brief candidate shorter than persistence interval	The candidate is shown as **CHECK**; it is not confirmed solely because of a short transient.	**PASS** as part of reported integrated tests.
Persistent candidate	A candidate remaining active for 250 ms sets the confirmed flame state and alerts when armed.	**PASS** as part of reported integrated tests.

Scenario	Expected Behaviour from Firmware	Reported Functional Outcome
Clear inputs return and remain stable	Clear requires **DO HIGH** and **AO** ≥ 850 for the 1,200 ms hold.	**PASS** as part of reported integrated tests.

This table summarizes the intended functional checks and the project's reported pass status. It does not claim a statistically measured false-trigger rate, an optical detection range or a calibrated response time. A future measurement campaign should record the module identifier, threshold adjustment, geometry, ambient illumination, time of each event and repeated observations before deriving such metrics.

5.3 Ultrasonic Distance and Zone Logic

The HC-SR04 verification uses a ruler-positioned reflecting target at selected distances. The logical checks are whether a valid echo is accepted, filtered distance is displayed, the zone flag enters at or below the configured 30 cm threshold, and recovery occurs only after the filtered distance remains above 33 cm for the configured clear hold. A no-echo condition must age into an unknown display state; it must not be interpreted as a measured clear distance.

Scenario	Expected Behaviour	Reported Functional Outcome
Target outside the entry boundary	No zone alert while the filtered distance remains above the active-entry threshold and no prior alert awaits clearing.	**PASS** as part of reported integrated tests.
Target crosses into the configured zone	Zone flag activates at filtered distance ≤ 30 cm; audible interval follows zone urgency logic.	**PASS** as part of reported integrated tests.
Target leaves the zone	The zone flag remains active until filtered distance is > 33 cm for 350 ms.	**PASS** as part of reported integrated tests.
Echo is absent or stale	Display presents **NO ECHO / ZONE UNKNOWN** according to input freshness; stale data is not presented as current clear data.	**PASS** as part of reported integrated tests.

The project report previously listed individual ruler and display values as well as absolute errors. Those exact figures have been removed because the available documentation did not provide the underlying repeatable measurement record needed to support them. The functional

pass result is retained, but it must not be taken to prove centimetre-level accuracy over the sensor's full operating range.

5.4 Integrated Verification Matrix

Test ID	Scenario	Expected Integrated Behaviour	Reported Result
IT-01	Armed, no active hazard	Green **LED** on when distance data is fresh; red **LED** and buzzer off; **OLED** shows clear status.	**PASS**
IT-02	Object enters configured zone	Zone alert becomes active and buzzer cadence follows proximity logic.	**PASS**
IT-03	Persistent flame-like response	Flame confirmation and alert occur after candidate persistence.	**PASS**
IT-04	Both sources active	Combined status is displayed; the shorter applicable alert period is selected.	**PASS**
IT-05	Short press during active alert	Current active source(s) are silenced; hazard status remains visible and monitoring continues.	**PASS**
IT-06	Long press of 2 seconds	System toggles **ARMED/DISARMED**; disarmed physical outputs are off.	**PASS**
IT-07	Hazard clears	Corresponding active flag clears only under configured clear rules; the old silence bit is removed.	**PASS**
IT-08	No ultrasonic echo	Stale range is shown as unknown rather than a current numeric distance.	**PASS**
IT-09	Power cycle	The stored event-start counter is recovered from **EEPROM**.	**PASS**

All nine integrated functional scenarios are reported as passed by the project team. This result supports a conclusion that the included functions were exercised successfully in the final integrated demonstration. It does not on its own establish a 100% reliability rate, a zero false-trigger rate per hour, sub-millisecond loop latency, a maximum distance error, or a statistically bounded repeatability figure. Those statements would require a recorded protocol, sufficient repeated trials and traceable measurements.

5.5 Discussion of Results

The main demonstrated outcome is functional integration: two separate sensing inputs affect a common alert model, the user can acknowledge currently active sources or disarm the annunciators, and the **OLED** communicates the interpretation. Persistence and hysteresis make the configured transitions less sensitive to single-sample changes than a raw direct-to-buzzer

circuit. The event counter provides a minimal persistent record of hazard-start transitions. These are architectural and functional benefits, rather than proof of certified hazard detection.

Several limitations shape the interpretation. First, analog values from a generic flame module are environment-dependent and are not calibrated units. Second, the HC-SR04 depends on a usable echo path and cannot identify the reflecting object. Third, the code's call to *pulseIn()* can block up to its timeout. Fourth, **EEPROM** only retains a count, not an event history. Fifth, there is no raw trial log in the supplied report to support strong statistical or metrological claims. The final report therefore reports successful functional tests and configured parameters while avoiding unsupported performance numbers.

5.6 Recommended Repeatable Test Record

For subsequent demonstrations or an expanded study, each test record should include date and firmware revision, sensor/module markings, threshold constants, test geometry, reference instrument, ambient conditions, measured values with units, number of repetitions, observed state transitions and the acceptance criterion. Performance claims should be computed only after the underlying observations are preserved. A distance-error statistic should state the reference distance and how error is calculated; false-trigger rate should state monitored duration and what counted as an unwanted confirmed event; response delay should use a defined time origin and a repeatable timing method.

These recommendations define additional work required before quantitative performance claims can be published. The present result is the reported end-to-end functional pass; no unsupported numerical metric is asserted.

5.7 Evidence Classification and Acceptance Interpretation

Statement Type	What This Report Can State	What it Does Not Claim
Functional test outcome	The project team reports that all nine integrated scenarios passed.	An independently witnessed test, certification, or statistical reliability result.
Firmware configuration	Thresholds and timing constants exactly as present in the supplied sketch.	That the values are optimal for every module, angle or ambient environment.
Illustrative diagram	A diagram that describes the documented logic or sample OLED text.	A photograph or measurement from the physical prototype.
Reported sample readings	Specific observations may be described only with their provenance and test conditions.	A calibrated performance specification without method, repetitions and uncertainty.
Future improvement	A proposed approach may be listed as future work.	A function that already exists or has been successfully validated.

This classification preserves a useful distinction between configuration, observed functionality and measured performance. It prevents a firmware constant from being mistaken

for an observed response, and prevents a pass/fail functional test from being converted into a numerical accuracy claim. The report follows this distinction throughout the results and conclusion.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

Luma Guard demonstrates a compact Arduino Nano-based monitoring prototype that combines a low-cost optical flame response with ultrasonic work-zone distance information. The design separates the meaning of each sensor, applies persistence and hysteresis to the configured state transitions, presents local status on an **OLED**, controls **LED** and buzzer annunciation, supports a single-button acknowledgement and arming interface, and stores a hazard-start count in **EEPROM**.

The completed integrated system is reported to have passed its included end-to-end functional tests. The most defensible result is that the documented functions operated as intended in the project demonstration. Numerical claims concerning precise distance error, false triggers per hour, statistical repeatability or exact timing performance are not retained without the corresponding raw measurement evidence. **6.2 Limitations**

- The flame sensor provides a module-dependent optical response and is vulnerable to ambient infrared energy, view-angle effects and comparator adjustment.
- The HC-SR04 requires a valid reflected echo and does not identify a person or the cause of a detected object.
- The **EEPROM** record stores a counter rather than a timestamped trace or full event history.
- The prototype is not certified fire-detection equipment and does not replace approved life-safety hardware or laboratory safety controls.

6.3 Future Enhancements

Future work can proceed in stages. **First**, capture a repeatable test log for controlled flame candidates, target distances, no-echo conditions and button operations; use the log to determine whether the present thresholds are appropriate for a specified environment. **Second**, reduce or characterize the blocking echo-measurement path through a carefully verified asynchronous or interrupt-assisted acquisition design. **Third**, evaluate an independently selected optical sensor with documented spectral response and consider a second sensing modality only if it improves a defined requirement. **Fourth**, improve event records if persistent history is useful, while considering **EEPROM** write endurance and a structured record format. **Fifth**, design a permanent **PCB** and enclosure only after the pin mapping, power requirements and electrical protections have been checked. Any future sensor such as a temperature module would be a new design revision and should not be described as included in the current build.

REFERENCES

The references below support the hardware, software-library and implementation descriptions. Exact module variants should be checked against the markings and documents supplied with the physical parts.

[1] Arduino, “Arduino Nano- technical specifications and documentation,” Arduino Store. Link: https://store.arduino.cc/products/arduino-nano

[2] Arduino, “Arduino Nano A000005 Datasheet,” board hardware reference. Link: https://docs.arduino.cc/resources/datasheets/A000005-datasheet.pdf

[3] Adafruit Industries, “Adafruit SSD1306,” open-source driver for monochrome SSD1306 OLED displays. Link: https://github.com/adafruit/Adafruit_SSD1306

[4] Adafruit Industries, “Adafruit GFX Library,” graphics primitives for supported displays. Link: https://github.com/adafruit/Adafruit-GFX-Library

[5] Arduino, “EEPROM Library,” built-in library documentation. Link: https://docs.arduino.cc/learn/built-in-libraries/eeprom/

[6] DFRobot, “Gravity: Analog Flame Sensor- Wiki and example code,” reference for an example analog flame-sensor module. Link: https://wiki.dfrobot.com/dfr0076/docs/186

[7] SparkFun Electronics, “HC-SR04 Ultrasonic Sensor,” module timing and distance calculation reference. Link: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[8] Arduino, “Select the right processor for Arduino Nano,” support guidance for processor selection. Link: https://support.arduino.cc/hc/en-us/articles/4401874304274

APPENDIX
FINAL FIRMWARE LISTING

The following listing records the final integrated firmware included in the supplied project report. Formatting has been normalized for readability while retaining the sketch logic and configured parameters.

Libraries required: Wire, EEPROM, Adafruit GFX and Adafruit SSD1306. The board and module pin assignments are listed in Chapter 3. Compile using the matching classic Nano board/processor selection and installed library set before any re-demonstration.

```
/*
  LumaGuard: Context-Aware Flame Event Detection and Work-Zone Monitoring
  Board: Arduino Nano ATmega328P (5 V classic Nano)
  Sensors: Flame sensor AO/DO + HC-SR04
  Interface: SSD1306 128x64 I2C OLED, red/green LEDs, buzzer, one button
  DHT22: intentionally excluded after repeated failed readings.
  Button: short press acknowledges currently active hazard source(s).
        long press (2 s) toggles ARMED/DISARMED.
  Note: educational prototype; not a certified fire alarm.
*/

#include <Wire.h>
#include <EEPROM.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ---------------- OLED ----------------
const uint8_t SCREEN_WIDTH = 128, SCREEN_HEIGHT = 64;
const int8_t OLED_RESET = -1;
const uint8_t OLED_ADDRESS = 0x3C;
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
bool oledReady = false;

// ---------------- Existing pin map ----------------
const uint8_t TRIG_PIN = 2, ECHO_PIN = 3;
const uint8_t GREEN_LED_PIN = 4, RED_LED_PIN = 5;
const uint8_t BUZZER_PIN = 6, BUTTON_PIN = 7;
const uint8_t FLAME_DO_PIN = 9, FLAME_AO_PIN = A0;

// ---------------- Calibration parameters ----------------
const float DISTANCE_ALERT_CM = 30.0;
const float DISTANCE_CLEAR_CM = 33.0;
const unsigned long ECHO_TIMEOUT_US = 25000UL;
```

```
const unsigned long DISTANCE_CLEAR_HOLD_MS = 350;
const unsigned long NO_ECHO_CLEAR_MS = 1200;
const unsigned long DISPLAY_ECHO_STALE_MS = 800;
const int FLAME_AO_TRIGGER = 800;
const int FLAME_AO_CLEAR = 850;
const unsigned long FLAME_CONFIRM_MS = 250;
const unsigned long FLAME_CLEAR_HOLD_MS = 1200;
const float FLAME_AO_CLEAR_REFERENCE = 1000.0;
const float FLAME_AO_STRONG_REFERENCE = 400.0;
const unsigned long BUTTON_DEBOUNCE_MS = 35;
const unsigned long LONG_PRESS_MS = 2000;
const unsigned long BUZZER_ON_MS = 80;
const unsigned int BUZZER_FREQUENCY_HZ = 2400;
const unsigned long DISTANCE_SLOW_PERIOD_MS = 900;
const unsigned long FLAME_SLOW_PERIOD_MS = 650;
const unsigned long FASTEST_PERIOD_MS = 180;
const unsigned long SENSOR_INTERVAL_MS = 100;
const unsigned long DISPLAY_INTERVAL_MS = 250;
const unsigned long SERIAL_INTERVAL_MS = 500;

// ---------------- Hazard bit masks ----------------
const uint8_t HAZARD_NONE = 0x00;
const uint8_t HAZARD_ZONE = 0x01;
const uint8_t HAZARD_FLAME = 0x02;

// ---------------- Sensor state ----------------
float distanceCm = -1.0, filteredDistanceCm = -1.0;
bool distanceFilterReady = false, distanceDanger = false;
bool haveValidDistance = false, distanceClearTimerActive = false;
unsigned long lastValidDistanceMs = 0, distanceClearStartedMs = 0;
uint8_t consecutiveNoEcho = 0;
int flameDigital = HIGH, flameAnalog = 1023;
bool flameRawCandidate = false, flameConfirmed = false;
bool flameCandidateTimerActive = false, flameClearTimerActive = false;
unsigned long flameCandidateStartedMs = 0, flameClearStartedMs = 0;

// ---------------- Alarm and button state ----------------
bool armed = true, alertSilenced = false;
uint8_t activeHazardMask = HAZARD_NONE, silencedHazardMask = HAZARD_NONE;
int lastRawButtonState = HIGH, stableButtonState = HIGH;
unsigned long lastButtonChangeMs = 0, buttonPressStartedMs = 0;
bool longPressHandled = false;

// ---------------- Buzzer state ----------------
bool buzzerToneOn = false, buzzerCycleActive = false;
unsigned long buzzerToneStartedMs = 0, lastBeepStartedMs = 0;

// ---------------- EEPROM event counter ----------------
const int EEPROM_ADDRESS = 0;
const uint16_t EEPROM_SIGNATURE = 0xF17E;
struct PersistentEventLog { uint16_t signature; uint32_t hazardStarts; };
```

```
PersistentEventLog eventLog;

// ---------------- Scheduler ----------------
unsigned long lastSensorReadMs = 0, lastDisplayUpdateMs = 0;
unsigned long lastSerialUpdateMs = 0;

void loadEventCounter() {
  EEPROM.get(EEPROM_ADDRESS, eventLog);
  if (eventLog.signature != EEPROM_SIGNATURE) {
    eventLog.signature = EEPROM_SIGNATURE;
    eventLog.hazardStarts = 0;
    EEPROM.put(EEPROM_ADDRESS, eventLog);
  }
}

void saveEventCounter() { EEPROM.put(EEPROM_ADDRESS, eventLog); }

void stopBuzzer() {
  noTone(BUZZER_PIN);
  buzzerToneOn = false;
  buzzerCycleActive = false;
}

float readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  unsigned long duration = pulseIn(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);
  if (duration == 0) return -1.0;
  float cm = duration * 0.0343 / 2.0;
  if (cm < 2.0 || cm > 400.0) return -1.0;
  return cm;
}

void updateDistanceState(float rawDistance, unsigned long now) {
  distanceCm = rawDistance;
  if (rawDistance >= 2.0 && rawDistance <= 400.0) {
    haveValidDistance = true;
    lastValidDistanceMs = now;
    consecutiveNoEcho = 0;
    if (!distanceFilterReady) {
      filteredDistanceCm = rawDistance;
      distanceFilterReady = true;
    } else {
      filteredDistanceCm = 0.60 * filteredDistanceCm + 0.40 * rawDistance;
    }
    if (!distanceDanger && filteredDistanceCm <= DISTANCE_ALERT_CM) {
      distanceDanger = true;
      distanceClearTimerActive = false;
```

```
      Serial.println(F("EVENT: ZONE ALERT STARTED"));
    }
    if (distanceDanger) {
      if (filteredDistanceCm > DISTANCE_CLEAR_CM) {
        if (!distanceClearTimerActive) {
          distanceClearTimerActive = true;
          distanceClearStartedMs = now;
        } else if (now - distanceClearStartedMs >= DISTANCE_CLEAR_HOLD_MS) {
          distanceDanger = false;
          distanceClearTimerActive = false;
          Serial.println(F("EVENT: ZONE ALERT CLEARED"));
        }
      } else distanceClearTimerActive = false;
    }
  } else {
    distanceClearTimerActive = false;
    if (consecutiveNoEcho < 255) consecutiveNoEcho++;
    if (distanceDanger && haveValidDistance && now - lastValidDistanceMs >=
NO_ECHO_CLEAR_MS) {
      distanceDanger = false;
      Serial.println(F("EVENT: DISTANCE ECHO LOST"));
    }
  }
}

void updateFlameState(unsigned long now) {
  flameDigital = digitalRead(FLAME_DO_PIN);
  flameAnalog = analogRead(FLAME_AO_PIN);
  flameRawCandidate = (flameDigital == LOW) || (flameAnalog <= FLAME_AO_TRIGGER);
  bool signalClear = (flameDigital == HIGH) && (flameAnalog >= FLAME_AO_CLEAR);
  if (flameRawCandidate) {
    flameClearTimerActive = false;
    if (!flameCandidateTimerActive) {
      flameCandidateTimerActive = true;
      flameCandidateStartedMs = now;
    }
    if (!flameConfirmed && now - flameCandidateStartedMs >= FLAME_CONFIRM_MS) {
      flameConfirmed = true;
      Serial.println(F("EVENT: FLAME CONFIRMED"));
    }
  } else {
    flameCandidateTimerActive = false;
    if (flameConfirmed && signalClear) {
      if (!flameClearTimerActive) {
        flameClearTimerActive = true;
        flameClearStartedMs = now;
      } else if (now - flameClearStartedMs >= FLAME_CLEAR_HOLD_MS) {
        flameConfirmed = false;
        flameClearTimerActive = false;
        Serial.println(F("EVENT: FLAME CONDITION CLEARED"));
      }
```

```
    } else flameClearTimerActive = false;
  }
}

void updateHazardMask() {
  uint8_t newMask = HAZARD_NONE;
  if (distanceDanger) newMask |= HAZARD_ZONE;
  if (flameConfirmed) newMask |= HAZARD_FLAME;
  uint8_t newlyActive = newMask & (uint8_t)(~activeHazardMask);
  if (newlyActive != HAZARD_NONE) {
    eventLog.hazardStarts++;
    saveEventCounter();
    Serial.print(F("HAZARD START COUNT: "));
    Serial.println(eventLog.hazardStarts);
  }
  activeHazardMask = newMask;
  silencedHazardMask &= activeHazardMask;
}

uint8_t getUnacknowledgedMask() {
  return activeHazardMask & (uint8_t)(~silencedHazardMask);
}

bool distanceReadingFresh() {
  return distanceFilterReady && (millis() - lastValidDistanceMs < DISPLAY_ECHO_STALE_MS);
}

const char* getFlameStatusText() {
  if (flameConfirmed) return "DETECTED";
  if (flameRawCandidate) return "CHECK";
  return "CLEAR";
}

const char* getAlertStatusText() {
  if (!armed) return "DISARMED";
  uint8_t pending = getUnacknowledgedMask();
  if (pending == (HAZARD_FLAME | HAZARD_ZONE)) return "FLAME + ZONE";
  if (pending & HAZARD_FLAME) return "FLAME ALERT";
  if (pending & HAZARD_ZONE) return "ZONE ALERT";
  if (activeHazardMask != HAZARD_NONE) return "SILENCED";
  if (flameRawCandidate) return "FLAME CHECK";
  if (!distanceReadingFresh()) return "ZONE UNKNOWN";
  return "SAFE";
}

void toggleArmMode() {
  armed = !armed;
  silencedHazardMask = HAZARD_NONE;
  stopBuzzer();
  Serial.println(armed ? F("SYSTEM: ARMED") : F("SYSTEM: DISARMED"));
}
```

```
void handleShortPress() {
  if (!armed) {
    Serial.println(F("Short press ignored: DISARMED"));
    return;
  }
  if (activeHazardMask != HAZARD_NONE) {
    silencedHazardMask |= activeHazardMask;
    stopBuzzer();
    Serial.println(F("ALERT ACKNOWLEDGED - monitoring continues"));
  } else Serial.println(F("No confirmed hazard to acknowledge"));
}

void handleButton(unsigned long now) {
  int raw = digitalRead(BUTTON_PIN);
  if (raw != lastRawButtonState) {
    lastRawButtonState = raw;
    lastButtonChangeMs = now;
  }
  if (now - lastButtonChangeMs >= BUTTON_DEBOUNCE_MS && raw != stableButtonState) {
    stableButtonState = raw;
    if (stableButtonState == LOW) {
      buttonPressStartedMs = now;
      longPressHandled = false;
    } else if (!longPressHandled) {
      unsigned long held = now - buttonPressStartedMs;
      if (held >= LONG_PRESS_MS) toggleArmMode();
      else handleShortPress();
    }
  }
  if (stableButtonState == LOW && lastRawButtonState == LOW && !longPressHandled &&
      now - buttonPressStartedMs >= LONG_PRESS_MS) {
    toggleArmMode();
    longPressHandled = true;
  }
}

unsigned long calculateDistancePeriod() {
  float d = filteredDistanceCm;
  if (d < 2.0) d = 2.0;
  if (d > DISTANCE_ALERT_CM) d = DISTANCE_ALERT_CM;
  float urgency = (DISTANCE_ALERT_CM - d) / (DISTANCE_ALERT_CM - 2.0);
  float period = DISTANCE_SLOW_PERIOD_MS - urgency * (DISTANCE_SLOW_PERIOD_MS -
FASTEST_PERIOD_MS);
  if (period < FASTEST_PERIOD_MS) period = FASTEST_PERIOD_MS;
  return (unsigned long)(period + 0.5);
}

unsigned long calculateFlamePeriod() {
  float response = (FLAME_AO_CLEAR_REFERENCE - (float)flameAnalog) /
    (FLAME_AO_CLEAR_REFERENCE - FLAME_AO_STRONG_REFERENCE);
```

```
  if (response < 0.0) response = 0.0;
  if (response > 1.0) response = 1.0;
  float period = FLAME_SLOW_PERIOD_MS - response * (FLAME_SLOW_PERIOD_MS -
FASTEST_PERIOD_MS);
  if (period < FASTEST_PERIOD_MS) period = FASTEST_PERIOD_MS;
  return (unsigned long)(period + 0.5);
}

unsigned long calculateBeepPeriod(uint8_t pending) {
  unsigned long period = 10000UL;
  if (pending & HAZARD_ZONE) period = calculateDistancePeriod();
  if (pending & HAZARD_FLAME) {
    unsigned long flamePeriod = calculateFlamePeriod();
    if (flamePeriod < period) period = flamePeriod;
  }
  return period;
}

void updateBuzzer(unsigned long now, uint8_t pending) {
  if (!armed || pending == HAZARD_NONE) {
    stopBuzzer();
    return;
  }
  unsigned long period = calculateBeepPeriod(pending);
  if (!buzzerCycleActive) {
    tone(BUZZER_PIN, BUZZER_FREQUENCY_HZ);
    buzzerToneOn = true;
    buzzerCycleActive = true;
    buzzerToneStartedMs = now;
    lastBeepStartedMs = now;
    return;
  }
  if (buzzerToneOn && now - buzzerToneStartedMs >= BUZZER_ON_MS) {
    noTone(BUZZER_PIN);
    buzzerToneOn = false;
  }
  if (!buzzerToneOn && now - lastBeepStartedMs >= period) {
    tone(BUZZER_PIN, BUZZER_FREQUENCY_HZ);
    buzzerToneOn = true;
    buzzerToneStartedMs = now;
    lastBeepStartedMs = now;
  }
}

void updateOutputs(unsigned long now) {
  uint8_t pending = getUnacknowledgedMask();
  if (!armed) {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
    stopBuzzer();
    return;
```

```
  }
  if (pending != HAZARD_NONE) {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, HIGH);
    updateBuzzer(now, pending);
    return;
  }
  if (activeHazardMask != HAZARD_NONE || flameRawCandidate) {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
    stopBuzzer();
    return;
  }
  if (!distanceReadingFresh()) {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
    stopBuzzer();
    return;
  }
  digitalWrite(GREEN_LED_PIN, HIGH);
  digitalWrite(RED_LED_PIN, LOW);
  stopBuzzer();
}

void updateOLED(unsigned long now) {
  if (!oledReady) return;
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextWrap(false);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.print(F("LUMAGUARD"));
  display.setCursor(78, 0);
  display.print(armed ? F("ARMED") : F("DISARMED"));
  display.drawFastHLine(0, 10, SCREEN_WIDTH, SSD1306_WHITE);
  display.setCursor(0, 13);
  display.print(F("FLAME: "));
  display.print(getFlameStatusText());
  display.setCursor(0, 23);
  display.print(F("DISTANCE"));
  display.setTextSize(2);
  display.setCursor(0, 33);
  bool validDisplay = distanceFilterReady && (now - lastValidDistanceMs <
DISPLAY_ECHO_STALE_MS);
  if (validDisplay) {
    display.print(filteredDistanceCm, 1);
    display.print(F(" cm"));
  } else display.print(F("NO ECHO"));
  display.setTextSize(1);
  display.setCursor(0, 54);
  display.print(F("ALERT: "));
```

```
  display.print(getAlertStatusText());
  display.display();
}


void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
  pinMode(FLAME_DO_PIN, INPUT); pinMode(FLAME_AO_PIN, INPUT);
  pinMode(GREEN_LED_PIN, OUTPUT); pinMode(RED_LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT); pinMode(BUTTON_PIN, INPUT_PULLUP);
  digitalWrite(TRIG_PIN, LOW);
  digitalWrite(GREEN_LED_PIN, LOW); digitalWrite(RED_LED_PIN, LOW);
  noTone(BUZZER_PIN);
  loadEventCounter();
  oledReady = display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS);
  if (oledReady) {
    display.clearDisplay(); display.setTextColor(SSD1306_WHITE);
    display.setTextSize(2); display.setCursor(0, 12);
    display.print(F("LUMAGUARD"));
    display.setTextSize(1); display.setCursor(0, 40);
    display.print(F("System starting...")); display.display();
  } else Serial.println(F("OLED init failed - check A4/A5, power and 0x3C"));
  unsigned long now = millis();
  lastSensorReadMs = now - SENSOR_INTERVAL_MS;
  lastDisplayUpdateMs = now; lastSerialUpdateMs = now;
  lastButtonChangeMs = now;
  Serial.println(F("LumaGuard started; DHT22 excluded"));
  Serial.println(F("Short press: acknowledge; long press: ARM/DISARM"));
}

void loop() {
  unsigned long now = millis();
  handleButton(now);
  if (now - lastSensorReadMs >= SENSOR_INTERVAL_MS) {
    lastSensorReadMs = now;
    updateDistanceState(readDistanceCm(), now);
    updateFlameState(now);
    updateHazardMask();
  }
  updateOutputs(millis());
  now = millis();
  if (now - lastDisplayUpdateMs >= DISPLAY_INTERVAL_MS) {
    lastDisplayUpdateMs = now;
    updateOLED(now);
  }
  if (now - lastSerialUpdateMs >= SERIAL_INTERVAL_MS) {
    lastSerialUpdateMs = now;
    printSerialStatus(now);
  }
}
```